

Milkshake Capstone API

1.0.0 OAS 3.0

API documentation for Capstone users - includes endpoints for advanced dairy farm management operations

Contact [CATTLEytics Support](#)

Servers

/

Authorize



Authentication

Login and authentication endpoints. Use POST /v1/auth with email and password to obtain a session cookie.



GET

/v1/auth/{id}



Get current session information

Retrieves the current authenticated user's session information, including:

- User identification (ID, name, email)
- Current site and role
- View mode (site vs advisor)
- Language preferences
- Administrative privileges

Response Details:

- All data is sourced from the server-side session (req.session)
- Does not query the database, returns cached session data

Common Use Cases:

- Verifying current authentication status
- Getting the current user's ID for API calls
- Checking which site the user is currently viewing
- Getting user display information (name, email)

Parameters

Try it out

Name	Description
id * required string (path)	id

Responses

Code	Description	Links
200	Ok Media type application/json Controls Accept header. Example Value Schema <pre>{ "authenticated": true, "userId": 123, "firstName": "John", "lastName": "Doe", "language": "en-us", "email": "john.doe@example.com", "siteId": 456, "role": "SiteAdministrator", "mode": "site", "isSuperAdmin": false, "punchclockLocation": "Milking Parlor" }</pre>	No links
401	Not authenticated Media type application/json Example Value Schema <pre>{ "title": "Not Found", "message": "Resource not found" }</pre>	No links

POST /v1/animals



Create a new animal

Creates a new animal record with the provided details.

Validation Rules:

- genderId (required): Must be a valid gender ID
- primaryTag (optional): If provided, must be unique within the site
- name (optional): Animal's name
- Other fields are optional and will be set if provided

Behavior:

- Animal is automatically associated with the authenticated user's site
- An "Arrival" event is automatically created with the current date/time
- If dim (days in milk) is provided, a "Fresh" event is created with back-dated eventDateTime
- Primary tag and name are stored as animal references (type 1 and 2 respectively)
- Additional animal references can be provided in the animalReferences array
- Animal update hook is triggered after creation to update related data
- Returns 400 if primary tag already exists for another animal in the same site

Response:

- Returns the created animal with related entities loaded (animalReferences, breed, currentPen)
- Includes signed image URL for secure access if image was provided
- Includes computed age from birth date

Parameters

Try it out

No parameters

Request body **required**

application/json

Example Value Schema

```
{
  "primaryTag": "1234",
  "name": "Bessie",
  "genderId": 1,
  "breedId": 5,
  "birthDate": "2022-03-15",
  "birthStatusId": 1,
  "currentPenId": 10,
  "homePenId": 10,
  "damId": 5000,
  "sireId": 100,
  "animalColorId": 2,
  "dim": 50,
  "lactationNumber": 1,
```

```
"imageUrl": "images/animal-1234.jpg",
"inDate": "2022-03-15",
"outDate": "2023-06-20",
"animalReferences": [
  {
    "reference": "string",
    "animalReferenceTypeId": 0
  }
]
```

Responses

Code	Description	Links
200	Ok	No links

Media type

application/json

Controls Accept header.

Examples

Example 1

Example Value Schema

```
{
  "genderId": 2,
  "primaryTag": "1234",
  "name": "Bessie",
  "breedId": 5,
  "birthDate": "2022-03-15",
  "currentPenId": 10,
  "homePenId": 10,
  "dim": 50,
  "lactationNumber": 1
}
```

201	Animal created successfully	No links
-----	-----------------------------	----------

Media type

application/json

Example Value Schema

```
{
  "id": 12345,
  "primaryTag": "1234",
  "name": "Bessie",
  "birthDate": "2022-03-15",
  "age": 2,
  "breed": {
    "name": "string",
    "id": 0
  },
}
```



Code	Description	Links
400	<pre>"gender": { "name": "string", "id": 0 }, "currentPen": { "name": "string", "id": 0 }, "homePen": { "name": "string", "id": 0 }, "dam": { "primaryTag": "string", "id": 0 }</pre> Invalid request data - primary tag already exists or required fields missing Media type application/json Example Value Schema <pre>{ "message": "Validation failed", "errors": [{ "field": "email", "message": "Email is required" }] }</pre>	No links
GET	/v1/animals/{id}	 
Get animal by ID Retrieves detailed information about a specific animal by their ID. Access Control: • Soft-deleted animals are excluded Response Details: • Includes all basic animal information (tags, name, birth date, etc.) • Includes related entities loaded eagerly: ◦ animalReferences: All tags and references for this animal ◦ breed: Breed information ◦ currentPen: Current pen assignment ◦ dam: Mother (dam) information ◦ sire: Father (sire) information ◦ birthStatus: Birth status code ◦ heatLatest: Most recent heat event ◦ breedLatest: Most recent breed event ◦ terminationCode: Termination code if animal is terminated		

- Computes age in years from birth date
- Generates signed image URL for secure access if image exists

Behavior:

- Returns 404 if animal not found, is soft-deleted, or belongs to different site

Parameters

Try it out

Name	Description
id * required number(\$double) (path)	id

Responses

Code	Description	Links
200	Ok Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 12345, "primaryTag": "1234", "name": "Bessie", "birthDate": "2022-03-15", "age": 2, "breed": { "name": "string", "id": 0 }, "gender": { "name": "string", "id": 0 }, "currentPen": { "name": "string", "id": 0 }, "homePen": { "name": "string", "id": 0 }, }</pre>	No links

Code	Description	Links
404	<pre>"dam": { "primaryTag": "string", }</pre> Animal not found or has been deleted Media type application/json Example Value Schema <pre>{ "title": "Not Found", "message": "Resource not found" }</pre>	No links
PATCH	/v1/animals/{id}	 
Update an animal Updates an existing animal with the provided fields. Only the fields included in the request body will be updated (partial update/PATCH semantics). Access Control: • Animal must belong to the authenticated user's site • Returns 404 if animal not found, deleted, or belongs to a different site Validation Rules: • All fields are optional (partial update) • primaryTag: Updates the primary tag reference • name: Updates the name reference • Other fields directly update the animal record Behavior: • If primaryTag is provided, updates or creates the primary tag reference (type 1) • If name is provided, updates or creates the name reference (type 2) • Animal update hook is triggered after update to refresh related data • Only updates fields that are explicitly provided in the request • Multi-tenant isolation enforced via siteld filter Response: • Returns the updated animal with all current values • Includes signed image URL for secure access if image exists • Includes computed age from birth date		
Parameters		Try it out

Name Description

id * required

number(\$double)
(path)

id

Request body required

application/json

Example Value Schema

```
{
  "primaryTag": "1234",
  "name": "Bessie",
  "breedId": 5,
  "birthDate": "2022-03-15",
  "birthStatusId": 1,
  "currentPenId": 10,
  "homePenId": 10,
  "damId": 5000,
  "sireId": 100,
  "animalColorId": 2,
  "genderId": 1,
  "imageUrl": "images/animal-1234.jpg",
  "inDate": "2022-03-15",
  "outDate": "2023-06-20",
  "lot": "LOT-2022-A"
}
```

Responses

Code Description Links

200 Ok No links

Media type

application/json

Controls Accept header.

Examples

Example 1

Example Value Schema

```
{
  "name": "Bessie Updated",
  "currentPenId": 12
}
```

404 Animal not found or does not belong to your site

No links

Code

Description

Links

Media type

application/json

Example Value Schema

```
{
  "title": "Not Found",
  "message": "Resource not found"
}
```

GET

/capstone/animals/by-reference/{primaryTag}



Get animal ID by reference tag

Looks up an animal by their primary tag reference and returns only the animal ID. This is optimized for simple ID lookups when you need to find an animal by tag before fetching full details via GET /v1/animals/:id.

Access Control:

- Soft-deleted animals are excluded

Behavior:

- First searches the animalReferences table for type 1 (primary tag)
- If not found, falls back to searching animal.primaryTag column
- Returns 404 if no animal found with that tag

Usage Pattern:

1. Call this endpoint with a primary tag to get the animal ID
2. Use the returned ID to call GET /v1/animals/:id for full animal details

Parameters

Try it out

Name

Description

primaryTag * required
string
(path)

The primary tag/identifier to search for

primaryTag

Responses

Code	Description	Links
200	Ok Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 12345}</pre>	No links
404	Animal not found with that reference tag Media type application/json Example Value Schema <pre>{ "title": "Not Found", "message": "Resource not found"}</pre>	No links

Animal Events Animal event tracking endpoints



POST	/v1/animal-events	 
Create a new animal event Creates one or more animal events with the provided details. Since animalIds is an array, this endpoint can create the same event for multiple animals simultaneously. Validation Rules: • animalIds (required): Array of animal IDs (must not be empty) • animalEventTypeId (required): Must be a valid event type ID • eventDateTime (required): ISO 8601 datetime string • For breeding events: sireId is required • For move events: penId is required • All animals must exist and not be soft-deleted • All related entities (pen, sire) must exist and belong to the user's site Behavior: • Creates one event record for each animal in the animalIds array • Events are automatically associated with the authenticated user's site • userId defaults to the authenticated user if not explicitly provided • For breeding events, sire name is automatically prepended to notes • Animal event hook is triggered after each event is created		

- Animal hook is triggered to update related animal data
- Returns 400 if any validation fails (animal not found, pen not found, sire not found for breeding)

Special Event Types:

- Breeding events require sireId and must exist in the site
- Move events should include penId
- Treatment events can include diagnosisRegimeId
- Calving events can include calvingAssistance, calvingDifficulty, calvingDueDate

Response:

- Returns the created event (for the first animal if multiple animals)
- Includes all related entities (animal, animalEventType, pen, sire, etc.)

Parameters

[Try it out](#)

No parameters

Request body required

[application/json](#)

Example Value [Schema](#)

```
{
  "animalIds": [
    12345,
    12346
  ],
  "animalEventTypeId": 5,
  "eventDateTime": "2025-11-03T10:30:00.000Z",
  "notes": "Animal treated for mastitis",
  "penId": 42,
  "sireId": 789,
  "breedingCodeId": 1,
  "diagnosisRegimeId": 10,
  "hipHeight": 145.5,
  "weight": 1250,
  "temperature": 101.5,
  "heatType": 2,
  "calvingAssistance": "Assisted",
  "calvingDifficulty": "Moderate",
  "calvingDueDate": "2025-08-15T00:00:00.000Z",
  "userId": 123,
  "eventDetail": [
    {
      "value": "string",
      "key": "string"
    }
  ]
}
```

Responses

Code	Description	Links
200	Ok Media type application/json Controls Accept header. Examples Example 1 Example Value Schema <pre>{ "animalIds": [12345], "animalEventTypeId": 5, "eventDateTime": "2025-11-03T10:30:00.000Z", "notes": "Treated for mastitis in rear left quarter", "diagnosisRegimeId": 10}</pre>	No links
400	Invalid request data - required fields missing, animal not found, pen not found, or sire not found Media type application/json Example Value Schema <pre>{ "message": "Validation failed", "errors": [{ "field": "email", "message": "Email is required" }]}</pre>	No links
GET /v1/animal-events 		
List animal events Retrieves a list of animal events with optional filtering. This simplified endpoint is designed for Capstone use cases where you need to filter by date range, animal, and event types. Query Parameters: - dateStart: Start date for filtering events (ISO 8601 format) - dateEnd: End date for filtering events (ISO 8601 format) - animalId: Filter events for a specific animal - animalEventTypesIds: Comma-separated list of event type IDs to filter by Behavior:		

- Only returns non-deleted events (soft delete pattern)
- Events are ordered by eventDateTime in ascending order by default
- Default limit is 1000 events
- Includes related entities: animal, animalEventType, diagnosisRegime, diagnosisRegimeDetail, pen, product, sire, user
- When dateStart and dateEnd are both provided, returns events between those dates (inclusive)
- When animalEventTypelds is an empty string, returns an empty array (useful for "select all" filters)
- When animalEventTypelds is provided, filters to only those event types

Special Behavior:

- When filtering by sireld, breeding events will include calf information to show breeding success
- This involves calculating expected calving dates and finding calves born within that window

Note: The actual implementation supports many more query parameters (limit, arrivals, repro, ship, treatments, pms, movements, activeTreatmentld, sireld) but this documentation only shows the core parameters for simplicity.

Parameters

Try it out

Name	Description
dateStart string (query)	dateStart
dateEnd string (query)	dateEnd
animalld number(\$double) (query)	animalld
animalEventTypelds string (query)	animalEventTypelds

Responses

Code	Description	Links
200	Ok Media type application/json Controls Accept header.	No links

Code

Description

Links

Example Value Schema

```
[
  {
    "id": 98765,
    "animalId": 12345,
    "animal": {
      "name": "string",
      "primaryTag": "string",
      "id": 0
    },
    "animalEventTypeId": 5,
    "animalEventType": {
      "name": "string",
      "id": 0
    },
    "eventDateTime": "2025-11-03T10:30:00.000Z",
    "notes": "Animal treated for mastitis in rear left quarter",
    "daysInMilk": 150,
    "lactationNumber": 2,
    "pen": {
      "name": "string",
      "id": 0
    },
    "penId": 42,
    "sire": {
```

DELETE /v1/animal-events/{id}



Delete an animal event

Soft-deletes an animal event by its ID. The event is marked as deleted but not physically removed from the database (soft delete pattern).

Access Control:

- Returns 404 if event not found, already deleted, or belongs to a different site

Behavior:

- Sets isDeleted flag to true
- Event will no longer appear in list queries
- Returns 204 No Content on success

Parameters

Try it out

Name	Description
------	-------------

id * required

number(\$double)
(path)

id

Responses

Code	Description	Links
------	-------------	-------

204

No content

No links

404

Animal event not found or does not belong to your site

No links

Media type

application/json

Example Value Schema

```
{
  "title": "Not Found",
  "message": "Resource not found"
}
```

Notes

Note management endpoints for tracking observations and actions



POST

/v1/notes



Create a new note

Creates a new note with the provided details. The note is automatically associated with the authenticated user's site and the creator is recorded.

Validation Rules:

- body: Can contain any text content (not validated for emptiness at API level)
- animalId: Optional, must be a valid animal ID if provided
- penId: Optional, must be a valid pen ID if provided
- tags: Optional, will be converted to lowercase automatically
- imageUrls: Optional, must be an array if provided
- assignedUserIds: Optional, all user IDs must belong to the same site

Behavior:

- Creator is automatically set to the authenticated user
- Site is automatically set to the authenticated user's site

- Tags are normalized to lowercase
- If assignedUserIds are provided, those users are assigned to the note
- Notifications are sent to newly assigned users
- If linked to an animal, triggers animal update hook

Response:

- Returns the created note with all related entities populated
- Includes signed image URLs for secure access
- Includes list of assigned users
- Includes assignment acknowledgment status

Parameters

Try it out

No parameters

Request body required

application/json

Example Value Schema

```
{
  "body": "Animal showing signs of lameness, needs follow-up examination",
  "animalId": 12345,
  "penId": 42,
  "tags": [
    "health",
    "lameness",
    "urgent"
  ],
  "imageUrls": [
    "images/note-12345.jpg",
    "images/note-12346.jpg"
  ],
  "expiryDate": "2025-12-31T23:59:59.000Z",
  "assignedUserIds": [
    101,
    102,
    103
  ]
}
```

Responses

Code	Description	Links
200	Ok	No links
Media type Examples		
application/json		

Code	Description	Links
	Controls Accept header. Example 1 Example Value Schema <pre>{ "body": "Animal 12345 showing signs of lameness in rear left leg. Scheduled veterinary examination for tomorrow.", "animalId": 12345, "tags": ["health", "lameness", "urgent"], "imageUrls": ["images/animal-12345-lameness-2025-11-03.jpg"], "assignedUserIds": [101, 102]}</pre>	
400	Invalid request data - assigned users do not belong to your site Media type application/json Example Value Schema <pre>{ "title": "Not Found", "message": "Resource not found"}</pre>	No links

Tasks

Task management endpoints for creating, updating, and tracking farm tasks

POST	/v1/tasks	 
Create a new task Creates a new task with the provided details. The task is automatically associated with the authenticated user's site and the creator is recorded. Validation Rules: - description (required): Task description, supports animal mentions using mentions format - taskId (required): Must be a valid category ID that belongs to the user's site - priority (optional): Defaults to "medium" if not specified - status (optional): Defaults to "pending" if not specified - completed (optional): Defaults to false; if true, completedDate is set to now - dueDate and startDate (optional): Must be valid ISO 8601 datetime strings		

- assignedUserId (optional): User to assign the task to; triggers notification if different from creator
- Scheduling fields (scheduleMode, scheduleRepeat, etc.) are only used when scheduleMode is "repeat"

Behavior:

- Animal IDs are extracted from mentions in the description (e.g., "mention 12345" links animal 12345)
- If task is marked completed, completedDate is set to current time and completedUserId defaults to creator
- Status history is initialized with the initial status and creator user ID
- If assignedUserId differs from the creator and notifications are enabled, the assignee receives a notification
- Repeat scheduling is only applied when scheduleMode is set to "repeat"

Response:

- Returns the created task with all related entities populated
- Dates are converted to the site's configured timezone
- Image URLs are signed for secure access

Parameters

Try it out

No parameters

Request body required

application/json

Example Value Schema

```
{
  "animalId": 12345,
  "assignedUserId": 789,
  "assignedUsersSecondaryIds": [
    101,
    102,
    103
  ],
  "completed": false,
  "completedUserId": 789,
  "description": "string",
  "instructions": "Perform visual inspection and record any abnormalities",
  "websiteUrl": "https://example.com/task-guide",
  "websiteAudioUrl": "https://example.com/audio/task-123.mp3",
  "websiteTestUrl": "https://example.com/quiz/task-123",
  "imageUrls": [
    "https://example.com/images/task1.jpg"
  ],
  "dueDate": "2025-12-31T23:59:59.000Z",
  "startDate": "2025-11-01T00:00:00.000Z",
  "taskCategoryId": 5,
  "priority": "high",
  "status": "completed",
  "notes": "This is a recurring monthly check",
  "toleranceDays": 3
}
```

Responses

Code

Description

Links

200

Ok

No links

Media type

Examples

application/json

Example 1

Controls Accept header.

Example Value Schema

```
{
  "description": "Monthly herd health check for animal @12345",
  "instructions": "Check for signs of mastitis and lameness",
  "taskCategoryId": 5,
  "priority": "high",
  "status": "pending",
  "completed": false,
  "dueDate": "2025-12-01T00:00:00.000Z",
  "assignedUserId": 789,
  "scheduleMode": "repeat",
  "scheduleRepeat": "monthly",
  "scheduleRepeatInterval": 1,
  "scheduleRepeatDayOfPeriod": 1,
  "notifyAssigner": true
}
```

201

Task created successfully

No links

Media type

application/json

Example Value Schema

```
{
  "id": 98765,
  "animalIds": [
    12345,
    12346
  ],
  "assignedUserId": 789,
  "completed": false,
  "completedDate": "2025-11-03T14:30:00.000Z",
  "completedUserId": 789,
  "description": "string",
  "instructions": "Perform visual inspection and record any abnormalities",
  "websiteUrl": "https://example.com/task-guide",
  "dueDate": "2025-12-31T23:59:59.000Z",
  "startDate": "2025-11-01T00:00:00.000Z",
  "taskCategoryId": 5,
  "priority": "high",
  "status": "completed",
  "notes": "This is a recurring monthly check",
  "userId": 123,
  "siteId": 1,
  "createdAt": "2025-11-03T10:00:00.000Z",
}
```

Code	Description	Links
400	<pre>"updatedAt": "2025-11-03T10:00:00.000Z"</pre> Invalid request data Media type application/json Example Value Schema <pre>{ "message": "Validation failed", "errors": [{ "field": "email", "message": "Email is required" }] }</pre>	No links
404	Task category not found or does not belong to your site Media type application/json Example Value Schema <pre>{ "title": "Not Found", "message": "Resource not found" }</pre>	No links

GET

/v1/tasks

List tasks

Retrieves a list of tasks filtered by the assigned user ID. This simplified endpoint is designed for Capstone use cases where you only need to filter by assignee.

Query Parameters:

- assignedUserId: Filter tasks assigned to a specific user

Behavior:

- Returns tasks scoped to the authenticated user's site (multi-tenant isolation)
- Only returns non-deleted tasks (soft delete pattern)
- Tasks are ordered by their order field in descending order
- Default limit is 25 tasks
- Includes related entities: assigned user, completed user, creator, task category, template
- Dates are converted to the site's configured timezone
- Image URLs are signed for secure access

Note: The actual implementation supports many more query parameters (animalIds, categoryId, dueDateStart, dueDateEnd, showCompleted, search, etc.) but this documentation only shows the

assignedUserId parameter for simplicity.

Parameters

Try it out

Name	Description
assignedUserId number(\$double) (query)	assignedUserId

Responses

Code	Description	Links
200	Ok	No links

Media type

application/json

Controls Accept header.

Example Value Schema

```
[
  {
    "id": 98765,
    "animalIds": [
      12345,
      12346
    ],
    "assignedUserId": 789,
    "completed": false,
    "completedDate": "2025-11-03T14:30:00.000Z",
    "completedUserId": 789,
    "description": "string",
    "instructions": "Perform visual inspection and record any abnormalities",
    "websiteUrl": "https://example.com/task-guide",
    "dueDate": "2025-12-31T23:59:59.000Z",
    "startDate": "2025-11-01T00:00:00.000Z",
    "taskCategoryId": 5,
    "priority": "high",
    "status": "completed",
    "notes": "This is a recurring monthly check",
    "userId": 123,
    "siteId": 1,
    "createdAt": "2025-11-03T10:00:00.000Z",
    "updatedAt": "2025-11-03T10:00:00.000Z"
  }
]
```



Get task by ID

Retrieves detailed information about a specific task by its ID.

Access Control:

- Task must belong to the authenticated user's site (multi-tenant isolation)
- Returns 404 if task not found, deleted, or belongs to a different site

Response Details:

- Includes all task fields with related entities (assigned user, completed user, creator, category, template)
- Dates are converted to the site's configured timezone
- Image URLs are signed for secure access
- Animal details are included if task is linked to animals

Parameters

[Try it out](#)

Name	Description
------	-------------

id * required
number(\$double)
(path)

Responses

Code	Description	Links
200	Ok	No links

Media type

application/json

Controls **Accept** header.

Example Value Schema

```
{
  "id": 98765,
  "animalIds": [
    12345,
    12346
  ],
  "assignedUserId": 789,
  "completed": false,
  "completedDate": "2025-11-03T14:30:00.000Z",
```

Code

Description

Links

```
"completedUserId": 789,
"description": "string",
"instructions": "Perform visual inspection and record any abnormalities",
"websiteUrl": "https://example.com/task-guide",
"dueDate": "2025-12-31T23:59:59.000Z",
"startDate": "2025-11-01T00:00:00.000Z",
"taskCategoryId": 5,
"priority": "high",
"status": "completed",
"notes": "This is a recurring monthly check",
"userId": 123,
"siteId": 1,
"createdAt": "2025-11-03T10:00:00.000Z",
"updatedAt": "2025-11-03T10:00:00.000Z",
},
```

404

Task not found or does not belong to your site

No links

Media type

application/json

Example Value Schema

```
{
  "title": "Not Found",
  "message": "Resource not found"
}
```

PATCH

/v1/tasks/{id}



Update task

Updates an existing task with the provided fields. Only the fields included in the request body will be updated (partial update/PATCH semantics).

Access Control:

- Task must belong to the authenticated user's site
- Returns 404 if task not found, deleted, or belongs to a different site

Validation Rules:

- All fields are optional (partial update)
- description: Task description text
- assignedUserId: User to assign the task to (null for unassigned)
- completed: Mark task as completed/incomplete
- completedUserId: User who completed the task
- dueDate/startDate: ISO 8601 datetime strings (can be null)
- priority: Must be valid TaskPriority value
- status: Must be valid TaskStatus value
- taskCategoryId: Must belong to the user's site
- imageUrls: Array of image URL strings
- All scheduling fields (scheduleMode, scheduleRepeat, etc.)

Behavior:

- Animal IDs are extracted from

Parameters

Try it out

Name	Description
------	-------------

id * required

number(\$double)
(path)

id

Request body required

application/json

Example Value Schema

```
{
  "animalId": 12345,
  "assignedUserId": 789,
  "assignedUsersSecondaryIds": [
    101,
    102,
    103
  ],
  "completed": false,
  "completedUserId": 789,
  "description": "string",
  "instructions": "Perform visual inspection and record any abnormalities",
  "websiteUrl": "https://example.com/task-guide",
  "websiteAudioUrl": "https://example.com/audio/task-123.mp3",
  "websiteTestUrl": "https://example.com/quiz/task-123",
  "imageUrls": [
    "https://example.com/images/task1.jpg"
  ],
  "dueDate": "2025-12-31T23:59:59.000Z",
  "startDate": "2025-11-01T00:00:00.000Z",
  "taskCategoryId": 5,
  "priority": "high",
  "status": "completed",
  "notes": "This is a recurring monthly check",
  "toleranceDays": 3
}
```

Responses

Code	Description	Links
200	Ok	No links
Media type		Examples

Code**Description****Links****application/json****Example 1**

Controls Accept header.

Example Value Schema

```
{
  "description": "Updated: Monthly herd health check for animal @12345",
  "completed": true,
  "status": "completed"
}
```

404

Task not found, task category invalid, or does not belong to your site

No links

Media type

application/json**Example Value** Schema

```
{
  "title": "Not Found",
  "message": "Resource not found"
}
```

DELETE**/v1/tasks/{id}****Delete task**

Soft-deletes a task by its ID. The task is marked as deleted but not physically removed from the database (soft delete pattern).

Access Control:

- Task must belong to the authenticated user's site
- Returns 404 if task not found, already deleted, or belongs to a different site

Behavior:

- Sets isDeleted flag to true
- Task will no longer appear in list queries
- Returns 204 No Content on success

Parameters**Try it out**

Name	Description
------	-------------

id * required

number(\$double)
(path)

id

Responses

Code	Description	Links
------	-------------	-------

204

No content

No links

404

Task not found or does not belong to your site

No links

Media type

application/json

Example Value Schema

```
{
  "title": "Not Found",
  "message": "Resource not found"
}
```

Users

User management and profile endpoints



GET

/v1/users



List users

Retrieves a list of users.

Query Parameters:

- limit` (number, default: 100): Maximum number of users to return

Response Details:

- Returns array of user objects

Parameters

Try it out

Name	Description
<code>_allSites</code> boolean (query)	--
<code>_limit</code> number(\$double) (query)	_limit
<code>_offset</code> number(\$double) (query)	_offset
<code>_sortField</code> string (query)	_sortField
<code>_sortDirection</code> string (query)	Available values : ASC, DESC --
<code>_search</code> string (query)	_search

Responses

Code	Description	Links
200	Ok Media type application/json Controls Accept header. Example Value Schema <pre>[{ "id": 123, "email": "john.doe@example.com", "firstName": "John", "lastName": "Doe", "language": "en-us", "phone": "+1-555-0123"</pre>	No links

Code	Description	Links
	<pre>}]</pre>	

GET	/v1/users/{id}	 
-----	----------------	---

Get user by ID

Retrieves detailed information about a specific user by their ID.

Response Details:

- Includes user's basic information (name, email, language, etc.)
- Soft-deleted users will return 404

Parameters

Try it out

Name	Description
------	-------------

id ★ required number (\$double) (path)	id
--	---------------------------------

Responses

Code	Description	Links
200	Ok	No links
Media type		
application/json		
Controls Accept header.		
Example Value Schema		
<pre>{ "id": 123, "email": "john.doe@example.com", "firstName": "John", "lastName": "Doe", "language": "en-us", }</pre>		

Code	Description	Links
	<pre>"phone": "+1-555-0123" }</pre>	
404	User not found or has been deleted	No links
	Media type application/json	
	Example Value Schema	
	<pre>{ "title": "Not Found", "message": "Resource not found" }</pre>	

Capstone

Endpoints available for Capstone users



GET

/capstone/animals/by-reference/{primaryTag}

^

Get animal ID by reference tag

Looks up an animal by their primary tag reference and returns only the animal ID. This is optimized for simple ID lookups when you need to find an animal by tag before fetching full details via GET /v1/animals/:id.

Access Control:

- Soft-deleted animals are excluded

Behavior:

- First searches the animalReferences table for type 1 (primary tag)
- If not found, falls back to searching animal.primaryTag column
- Returns 404 if no animal found with that tag

Usage Pattern:

- Call this endpoint with a primary tag to get the animal ID
- Use the returned ID to call GET /v1/animals/:id for full animal details

Parameters

Try it out

Name	Description
primaryTag * required string (path)	The primary tag/identifier to search for primaryTag

Responses

Code	Description	Links
200	Ok Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 12345 }</pre>	No links
404	Animal not found with that reference tag Media type application/json Example Value Schema <pre>{ "title": "Not Found", "message": "Resource not found" }</pre>	No links

Schemas



```
UserResponse {
  description: User response model

  id*
    number($double)
    example: 123
    Unique identifier for the user

  email*
    string
    example: john.doe@example.com
    User's email address

  firstName*
    string
    example: John
    User's first name

  lastName*
    string
    example: Doe
    User's last name

  language*
    string
    example: en-us
    User's preferred language (ISO 639-1 code)

  phone
    string
    example: +1-555-0123
    User's phone number
}
```

```
ErrorResponse {
  description: Common error response model

  title
    string
    example: Not Found
    Error title

  message*
    string
    example: Resource not found
    Error message
}
```

TaskPriority string

WARNING!

DO NOT EDIT THIS FILE

CHANGES WILL BE OVERWRITTEN!

Make changes in packages/shared and run npm run copy-shared-to-packages

Enum:

[high, highest, low, lowest, medium]

TaskStatus string

WARNING!

DO NOT EDIT THIS FILE

CHANGES WILL BE OVERWRITTEN!

Make changes in packages/shared and run npm run copy-shared-to-packages

Enum:

[completed, in-progress, pending, scheduled, skipped]

```

TaskResponse    {
  description:    Response model for a created task

  id*            [...]
  animalIds*     [...]
  assignedUserId [...]
  completed*     [...]
  completedDate  [...]
  completedUserId [...]
  description*   [...]
  instructions   [...]
  websiteUrl     [...]
  dueDate        [...]
  startDate      [...]
  taskCategoryId* [...]
  priority*      TaskPriority  [...]
  status*        TaskStatus   [...]
  notes          [...]
  userId*        [...]
  siteId*        [...]
  createdAt*     [...]
  updatedAt*     [...]
}

```

```

ValidationErrorResponse {
  description:    Validation error response model

  message*       [...]
  errors         [...]
}

```

```

TaskMode    string
Enum:
  Array [ 2 ]

```

```

TaskRepeat    string
Enum:
  Array [ 4 ]

```

```

TaskCreateRequest  {
    description:          Request body for creating a new task

    animalId              [...]
    assignedUserId        [...]
    assignedUsersSecondaryIds  [...]
    completed             [...]
    completedUserId       [...]
    description*          [...]
    instructions          [...]
    websiteUrl            [...]
    websiteAudioUrl       [...]
    websiteTestUrl        [...]
    imageUrls             [...]
    dueDate               [...]
    startDate             [...]
    taskCategoryId*       [...]
    priority              TaskPriority    [...]
    status                TaskStatus     [...]
    notes                [...]
    toleranceDays         [...]
    notifyAssigner        [...]
    scheduleEndDate       [...]
    scheduleMode          TaskMode       [...]
    scheduleReset         [...]
    scheduleRepeat        TaskRepeat    [...]
    scheduleRepeatInterval  [...]
    scheduleRepeatDayOfWeek  [...]
    scheduleRepeatDayOfPeriod  [...]
    templateId            [...]
}

```

```

Partial_TaskCreateRequest_ {
  description:           Make all properties in T optional

  animalId               [...]
  assignedUserId          [...]
  assignedUsersSecondaryIds [...]
  completed               [...]
  completedUserId         [...]
  description             [...]
  instructions            [...]
  websiteUrl              [...]
  websiteAudioUrl         [...]
  websiteTestUrl          [...]
  imageUrls               [...]
  dueDate                 [...]
  startDate               [...]
  taskCategoryId          [...]
  priority                TaskPriority  [...]
  status                  TaskStatus   [...]
  notes                   [...]
  toleranceDays           [...]
  notifyAssigner          [...]
  scheduleEndDate         [...]
  scheduleMode            TaskMode     [...]
  scheduleReset           [...]
  scheduleRepeat          TaskRepeat   [...]
  scheduleRepeatInterval  [...]
  scheduleRepeatDayOfWeek [...]
  scheduleRepeatDayOfPeriod [...]
  templateId              [...]
}

```

```
NoteResponse {
  description:      Response model for a created note

  id*               [...]
  body*             [...]
  animalId         [...]
  penId            [...]
  tags             [...]
  imageUrls        [...]
  imageUrlsSigned  [...]
  expiryDate       [...]
  userId*          [...]
  siteId*          [...]
  createdAt*       [...]
  updatedAt*       [...]
  assignedUsers    [...]
}
```

```
NoteCreateRequest {
  description:      Request body for creating a new note

  body*            [...]
  animalId         [...]
  penId            [...]
  tags             [...]
  imageUrls        [...]
  expiryDate       [...]
  assignedUserIds  [...]
}
```

```
AuthSessionResponse {
  description: Authentication session response model

  authenticated*      [...]
  userId*             [...]
  firstName*          [...]
  lastName*           [...]
  language*           [...]
  email*              [...]
  siteId*             [...]
  role*               [...]
  mode*               [...]
  isSuperAdmin*       [...]
  punchclockLocation  [...]
}
```

```
AnimalResponse {
  description:      Animal response model

  id*              [...]
  primaryTag       [...]
  name             [...]
  birthDate        [...]
  age              [...]
  breed            [...]
  gender*          [...]
  currentPen       [...]
  homePen          [...]
  dam              [...]
  sire             [...]
  birthStatus      [...]
  heatLatest       [...]
  breedLatest      [...]
  terminationCode [...]
  daysInMilk       [...]
  lactationNumber  [...]
  isCalf           [...]
  isCow            [...]
  isHeifer         [...]
  isBull           [...]
  imageUrl         [...]
  imageUrlSigned   [...]
  siteId*          [...]
  createdAt*       [...]
  updatedAt*       [...]
  animalReferences [...]
}
```

```

AnimalCreateRequest {
  description: Request body for creating a new animal

  primaryTag
      string
      example: 1234
      Primary tag/identifier for the animal

  name
      string
      example: Bessie
      Animal's name

  genderId*
      integer($int32)
      example: 1
      Gender ID

  breedId
      integer($int32)
      example: 5
      Breed ID

  birthDate
      string
      example: 2022-03-15
      Birth date (ISO 8601 date format)

  birthStatusId
      integer($int32)
      example: 1
      Birth status ID

  currentPenId
      integer($int32)
      example: 10
      Current pen ID

  homePenId
      integer($int32)
      example: 10
      Home pen ID

  damId
      integer($int32)
      example: 5000
      Dam (mother) ID

  sireId
      integer($int32)
      example: 100
      Sire (father) ID

  animalColorId
      integer($int32)
      example: 2
      Animal color ID

  dim
      integer($int32)
      example: 50
      Days in milk (DIM) at arrival

  lactationNumber
      integer($int32)
      example: 1
      Lactation number at arrival

  imageUrl
      string
      example: images/animal-1234.jpg
      Image URL for the animal

  inDate
      string
      example: 2022-03-15

```

example: 2022-05-15

Date the animal entered the facility

outDate

string

example: 2023-06-20

Date the animal left the facility

animalReferences

[

Additional animal references (tags)

{...}]

}

AnimalUpdateRequest	{
description:	Request body for updating an animal (partial update)
primaryTag	string example: 1234 Primary tag/identifier for the animal
name	string example: Bessie Animal's name
breedId	integer(\$int32) example: 5 Breed ID
birthDate	string example: 2022-03-15 Birth date (ISO 8601 date format)
birthStatusId	integer(\$int32) example: 1 Birth status ID
currentPenId	integer(\$int32) example: 10 Current pen ID
homePenId	integer(\$int32) example: 10 Home pen ID
damId	integer(\$int32) example: 5000 Dam (mother) ID
sireId	integer(\$int32) example: 100 Sire (father) ID
animalColorId	integer(\$int32) example: 2 Animal color ID
genderId	integer(\$int32) example: 1 Gender ID
imageUrl	string example: images/animal-1234.jpg Image URL for the animal
inDate	string example: 2022-03-15 Date the animal entered the facility
outDate	string example: 2023-06-20 Date the animal left the facility
lot	string example: LOT 2022 A

example: L01-2022-1
Lot the animal belongs to

}

AnimalIdResponse {
 description: Response containing only the animal ID
 id* **number(\$double)**
 example: 12345
 Unique identifier for the animal
}

```

AnimalEventResponse {
  description:      Animal event response model

  id*
    number($double)
    example: 98765
    Unique identifier for the animal event

  animalId*
    number($double)
    example: 12345
    ID of the animal this event belongs to

  animal*
    {
      description:      Animal information

      name
        string
      primaryTag
        string
      id*
        number($double)
    }

  animalEventTypeId*
    number($double)
    example: 5
    ID of the event type

  animalEventType*
    {
      description:      Event type information

      name*
        string
      id*
        number($double)
    }

  eventDateTime*
    string
    example: 2025-11-03T10:30:00.000Z
    Date and time when the event occurred (ISO 8601 format)

  notes
    string
    example: Animal treated for mastitis in rear left quarter
    Notes about the event

  daysInMilk
    number($double)
    example: 150
    Days in milk (DIM) at the time of the event

  lactationNumber
    number($double)
    example: 2
    Lactation number at the time of the event

  pen
    {
      description:      Pen information (for move
                        events or pen-related events)

      name*
        string
      id*
        number($double)
    }

  penId
    number($double)
    example: 42
    Pen ID

  sire
    {
      description:      Sire information (for breeding
                        events)

```

	name string
	id* number(\$double)
sireId	number(\$double) example: 789 Sire ID (for breeding events)
diagnosisRegime	description: Diagnosis regime information (for treatment events) name* string id* number(\$double)
diagnosisRegimeId	number(\$double) example: 10 Diagnosis regime ID
diagnosisRegimeDetail	description: Diagnosis regime detail information (for treatment events) name* string id* number(\$double)
diagnosisRegimeDetailId	number(\$double) example: 25 Diagnosis regime detail ID
product	description: Product information (for treatment events) name* string id* number(\$double)
productId	number(\$double) example: 15 Product ID (for treatment events)
productDosage	number(\$double) example: 10.5 Product dosage in mL (for treatment events)
activeTreatmentId	number(\$double) example: 500 Active treatment ID (if event is part of an active treatment)
user	description: User who created/recorded the event email* string lastName* string firstName* string

```

        id*
        number($double)
    }

    heatType
        number($double)
        example: 2
        Heat type (for heat events) 1 = Coming into heat, 2 =
        Standing heat, 3 = Going out of heat

    hipHeight
        number($double)
        example: 145.5
        Hip height in cm (for measurement events)

    weight
        number($double)
        example: 1250
        Weight in pounds (for measurement events)

    temperature
        number($double)
        example: 101.5
        Temperature in Fahrenheit (for health check events)

    calvingAssistance
        string
        example: Assisted
        Calving assistance (for calving events)

    calvingDifficulty
        string
        example: Moderate
        Calving difficulty (for calving events)

    calvingDueDate
        string
        example: 2025-08-15T00:00:00.000Z
        Calving due date (for breeding/pregnancy events)

    calves
        [
        Calves born (for breeding events, only populated when
        sireId filter is used)

        {
            birthDate
                [...]
            primaryTag
                [...]
            id*
                [...]
        }
    ]

    siteId*
        number($double)
        example: 1
        ID of the site this event belongs to

    createdAt*
        string
        example: 2025-11-03T10:00:00.000Z
        Timestamp when the event record was created

    updatedAt*
        string
        example: 2025-11-03T10:00:00.000Z
        Timestamp when the event record was last updated
}

```



```

AnimalEventCreateRequest {
  description: Request body for creating a new animal event

  animalIds*
    [
      example: [ 12345, 12346 ]
      Array of animal IDs this event applies to

      [...]]

  animalEventTypeId*
    integer($int32)
    example: 5
    Event type ID

  eventDateTime*
    string
    example: 2025-11-03T10:30:00.000Z
    Date and time when the event occurred (ISO 8601 format)

  notes
    string
    example: Animal treated for mastitis
    Notes about the event

  penId
    integer($int32)
    example: 42
    Pen ID (for move events or pen-related events)

  sireId
    integer($int32)
    example: 789
    Sire ID (for breeding events)

  breedingCodeId
    integer($int32)
    example: 1
    Breeding code ID (for breeding events)

  diagnosisRegimeId
    integer($int32)
    example: 10
    Diagnosis regime ID (for treatment events)

  hipHeight
    number($double)
    example: 145.5
    Hip height in cm (for measurement events)

  weight
    number($double)
    example: 1250
    Weight in pounds (for measurement events)

  temperature
    number($double)
    example: 101.5
    Temperature in Fahrenheit (for health check events)

  heatType
    integer($int32)
    example: 2
    Heat type (for heat events) 1 = Coming into heat, 2 =
    Standing heat, 3 = Going out of heat

  calvingAssistance
    string
    example: Assisted
    Calving assistance (for calving events)

  calvingDifficulty
    string
    example: Moderate
    Calving difficulty (for calving events)

```

```
calvingDueDate      string
                    example: 2025-08-15T00:00:00.000Z
                    Calving due date (for breeding/pregnancy events, ISO 8601
                    format)

userId              integer($int32)
                    example: 123
                    User ID who created the event (defaults to authenticated
                    user if not provided)

eventDetail         [
                    Event details (key-value pairs for additional data)

                    {
                      value*      string
                      key*        string
                    }
  ]

}
```